HashiCorp Certified: Terraform Associate（004）　目標 4c・4d　#07

変数・出力と型

variable ブロックの引数、値の渡し方と優先順位（-var・tfvars・TF_VAR_・default）、output と terraform output、ローカル値、プリミティブ・コレクション・構造型、optional、型の変換と any

各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Use input variables to add module arguments・variable block reference・output block reference・Use locals to reuse expressions・Type Constraints・Command: output）を書いた。英語の原文は『』で引用した

第1章

入力・出力・ローカル

引数、戻り値、関数の中の変数。

モジュールを関数にたとえると

variable = 引数
外から値を受け取る。var.NAME

↓

locals = 関数の中の変数
式に名前を付ける。local.NAME（参照は単数形）

↓

output = 戻り値
親・CLI・remote state・自動化ツールへ。module.M.NAME

```
variable "environment" {
  type        = string
  description = "Deployment environment name"
  default     = "dev"            # リテラルだけ
  validation {
    condition     = contains(["dev","staging","prod"], var.environment)
    error_message = "Environment must be dev, staging, or prod."
  }
}
locals {
  name = "${var.project}-${var.environment}"
}
output "web_ip" {
  value       = aws_instance.web.public_ip   # 必須
  description = "Public IP of the web server"
}
```

出典：variable block reference／output block reference／Use locals to reuse expressions

variable ブロックの引数

引数	意味	覚えどころ
type	型の制約	書かなければ any
default	既定値	なければ必須（plan の前に入力を求める）。リテラルだけ、参照不可
description	使う人向けの説明	保守のメモはコメントに
validation	condition・error_message	計画を作るときに評価、false で止まる
sensitive	CLI の出力で隠す	使った式も sensitive。ステートには残る
nullable	null を入れてよいか	既定 true。false なら null 不可
ephemeral	ステート・プランに残さない	#10

> 変数名に使えない：source・version・providers・count・for_each・lifecycle・depends_on・locals

出典：variable block reference

値の渡し方と優先順位

1. -var・-var-file（書いた順）・HCP Terraform
   いちばん強い
2. *.auto.tfvars・*.auto.tfvars.json
   名前の順
3. terraform.tfvars.json
4. terraform.tfvars
5. 環境変数 TF_VAR_name
6. default
   いちばん弱い

```
$ terraform apply -var="instance_type=t3.medium"
$ terraform apply -var-file="prod.tfvars"   # 自動では読まれない名前
$ export TF_VAR_environment=staging
$ export TF_VAR_cfg='{"key": "value", "list": ["a", "b"]}'
```

自動で読まれる：terraform.tfvars・terraform.tfvars.json・*.auto.tfvars・*.auto.tfvars.json

子モジュールの変数は、親の module ブロックの引数で渡す。

秘密の値が入った tfvars は VCS に入れない。

出典：Use input variables to add module arguments（Assign values to variables）

output と terraform output

output の引数	意味
value（必須）	返す値。ステートに保存
description	使う人向けの説明
sensitive	plan・apply で (sensitive value)
ephemeral	ステートに残さない（子モジュールの出力）
depends_on・precondition	明示の依存・事前の条件

```
$ terraform output              # 全部（sensitive は隠す）
$ terraform output lb_address   # 1 つ
$ terraform output -json | jq   # JSON
$ terraform output -raw lb_address
# 文字列そのまま（string・number・bool だけ）
```

-json・-raw では sensitive も平文で出る。

sensitive な属性を value に使うと、output にも sensitive = true が必要。

出典：output block reference／Command: output

第2章

型

同じ型の集まりか、ちがう型の組か。

型の全体像

種類	型	特徴
プリミティブ	string・number・bool	number は整数も小数も。15 ⇔ "15"、true ⇔ "true" は自動変換
コレクション	list(T)	0 からの番号。同じ型の要素
コレクション	map(T)	文字列のキー。同じ型の値
コレクション	set(T)	順番もキーもなく、重複しない
構造	object({k = T, …})	名前付きの属性ごとに型。キーは全部必要（多い分は捨てる）
構造	tuple([T, …])	位置ごとに型。数はぴったり
動的	any	型ではなく置き場所。『very rarely the correct type constraint』

> list・map だけ書くと list(any)・map(any)。新しいコードは完全な形で。

出典：Type Constraints

型の変換と optional

渡した値 → 型	結果
["a", 15, true] → list(string)	["a", "15", "true"]
["b", "a", "b"] → set(string)	重複が消え、順番もなくなる
["a", 1, "b"] → list(any)	list(string)：["a", "1", "b"]
["a", [], "b"] → list(any)	エラー（そろう型がない）
{name = [...], age = 12} → map(string)	エラー（リストは文字列にならない）

```
variable "with_optional_attribute" {
  type = object({
    a = string                # 必須
    b = optional(string)      # 省くと null
    c = optional(number, 127) # 省くと 127
  })
}
```

> null でない既定値は、省いても null を渡しても既定値になる。

> any を使ってよいのは、中を見ずに jsonencode などでそのまま渡すときだけ。

出典：Type Constraints（Conversion of Complex Types／any／Optional Object Type Attributes）

本番での問われ方

ねらい	見るところ
variable	var. で参照、default がないと必須、type の既定は any、予約語
validation・sensitive	計画で評価して止まる、sensitive は表示だけ
優先順位	-var ＞ auto.tfvars ＞ tfvars.json ＞ tfvars ＞ TF_VAR_ ＞ default
output	value 必須、-json・-raw は平文、-raw は単純な型だけ
locals	locals で定義・local で参照、外から渡せない
型	コレクションと構造のちがい、変換の結果、optional の既定値、any の使いどころ

出典：Exam Content List（004）4c・4d／本資料の問題演習（全 30 問）